

El Protocolo de Control de Transmisión (TCP): Fundamentos, Mecanismos y Confiabilidad

Resumen Ejecutivo

El Protocolo de Control de Transmisión (TCP) es la piedra angular de las comunicaciones confiables en la arquitectura de Internet. Diseñado para operar sobre sistemas de conmutación de paquetes inherentemente no confiables —donde los datos pueden perderse, duplicarse o desordenarse—, TCP proporciona a las aplicaciones una abstracción de flujo de datos continuo y sin errores. Sus pilares fundamentales incluyen la técnica de **Confirmación Positiva con Retransmisión (PAR)** y el **paradigma de la ventana deslizante**, los cuales optimizan el rendimiento al permitir múltiples paquetes "en vuelo". Mediante algoritmos adaptativos como el de **Karn**, el **inicio lento (slow-start)** y el **incremento aditivo con disminución multiplicativa (AIMD)**, TCP no solo garantiza la integridad de la entrega, sino que gestiona dinámicamente el flujo y la congestión de la red para evitar el colapso sistémico.

1. Naturaleza y Necesidad del Servicio Confiable

En los niveles inferiores, las redes de computadoras ofrecen una entrega de paquetes no confiable debido a errores de transmisión, fallos de hardware o sobrecargas que resultan en rutas dinámicas cambiantes. TCP surge como una solución de propósito general para evitar que cada programador de aplicaciones deba implementar su propia detección y recuperación de errores.

Propiedades clave del servicio TCP:

- **Orientado a flujo (Stream):** Los datos se perciben como una secuencia continua de octetos. El receptor obtiene exactamente la misma secuencia enviada por el origen.
- **Conexión de circuito virtual:** Antes de la transferencia, los puntos finales deben acordar el establecimiento de una conexión. Se denomina "virtual" porque actúa como un circuito dedicado, aunque use paquetes.
- **Transferencia con buffer:** TCP recolecta datos del flujo para llenar paquetes de tamaño razonable, optimizando el tráfico de red. Incluye un mecanismo de **empuje (push)** para forzar la transferencia inmediata si la aplicación lo requiere.
- **Flujo no estructurado:** El protocolo no identifica el contenido (ej. registros de nómina); las aplicaciones deben acordar el formato del flujo previamente.
- **Comunicación Full Duplex:** Permite la transferencia concurrente en ambas direcciones mediante dos flujos independientes. Utiliza el **piggybacking** para enviar información de control de un flujo en los paquetes de datos del flujo opuesto.

2. Mecanismos de Transmisión Confiable

Confirmación Positiva con Retransmisión (PAR)

Es la técnica base para la confiabilidad. El emisor inicia un temporizador al enviar un paquete. Si la confirmación (ACK) llega antes de que expire, el temporizador se cancela; de lo contrario, el paquete se retransmite. Para resolver la duplicación de paquetes (causada por retrasos excesivos), TCP asigna **números de secuencia** únicos a cada paquete, permitiendo al receptor descartar duplicados.

El Paradigma de la Ventana Deslizante

A diferencia del protocolo simple de "enviar y esperar" (que desperdicia capacidad de red), la ventana deslizante permite enviar múltiples paquetes antes de recibir una confirmación.

- **Rendimiento:** Mantiene la red saturada de paquetes, aumentando sustancialmente el *throughput*.
- **Estados de los paquetes:** La ventana divide la secuencia en tres: paquetes ya confirmados (izquierda), paquetes en tránsito (dentro de la ventana) y paquetes aún no enviados (derecha).

3. Arquitectura y Abstracción de Conexión

Puntos Finales y Puertos

TCP utiliza el concepto de **conexión** como su abstracción fundamental, identificada por un par de **puntos finales (endpoints)**.

- **Definición de punto final:** Es un par (Dirección IP, Puerto).
- **Multiplexación:** Un solo puerto TCP puede ser compartido por múltiples conexiones simultáneas en la misma máquina, ya que TCP usa ambos puntos finales para identificar el mensaje entrante.
- **Puertos Reservados:** Existen "puertos bien conocidos" (ej. 80 para WWW, 25 para SMTP, 53 para DNS) asignados para servicios comunes.

El Segmento TCP

La unidad de transferencia es el **segmento**, que consta de un encabezado (mínimo 20 octetos) y una carga útil (*payload*).

Campo	Función
Source/Destination Port	Identifican las aplicaciones en los extremos.
Sequence Number	Posición de los datos en el flujo del emisor.
Acknowledgement Number	Número del siguiente octeto que el receptor espera.
Window	Anuncia cuánto espacio de buffer tiene disponible el receptor.
Code Bits	Bits de control (SYN, ACK, FIN, RST, PSH, URG).
Checksum	Verifica la integridad del encabezado y los datos usando un pseudo-encabezado IP.

4. Gestión de Flujo y Control de Congestión

Control de Flujo Extremo a Extremo

TCP utiliza una **ventana de tamaño variable**. El receptor envía "anuncios de ventana" que indican su capacidad actual de buffer. Si el buffer se llena, el receptor anuncia una ventana de cero para detener el flujo hasta que haya espacio disponible.

Adaptación al Retraso (Algoritmo de Karn)

TCP monitorea el tiempo de ida y vuelta (RTT) para ajustar sus *timeouts*. Debido a la **ambigüedad de confirmación** (no saber si un ACK es del paquete original o de una retransmisión), se aplica el **Algoritmo de Karn**:

1. No se actualiza la estimación de RTT con segmentos retransmitidos.
2. Se utiliza una estrategia de **retroceso del temporizador (timer backoff)**, duplicando el timeout con cada retransmisión fallida.

Estrategias contra la Congestión

TCP asume que la pérdida de paquetes se debe a la congestión. Utiliza dos límites: la ventana del receptor y la **ventana de congestión**.

- **Disminución Multiplicativa:** Al perderse un segmento, la ventana de congestión se reduce a la mitad.
- **Inicio Lento (Slow-Start):** Al iniciar o recuperarse de congestión, la ventana comienza en un segmento y aumenta en uno por cada ACK recibido.
- **AIMD (Incremento Aditivo, Disminución Multiplicativa):** Una vez alcanzado un umbral, el crecimiento de la ventana se vuelve lineal para evitar saturar la red.

5. Políticas de Gestión de Colas en Enrutadores

Las capas inferiores afectan el rendimiento de TCP a través de la gestión de colas en los enrutadores:

- **Tail-drop (Descarte por la cola):** Descarta paquetes nuevos si la cola está llena. Esto provoca **sincronización global**, donde múltiples conexiones entran en inicio lento simultáneamente, reduciendo la eficiencia de la red.
- **RED (Detección Temprana Aleatoria):** El enrutador descarta paquetes de forma aleatoria antes de que la cola se llene. Esto evita la sincronización global y permite manejar ráfagas cortas de tráfico sin penalizar a todas las conexiones.

6. Ciclo de Vida de la Conexión

Establecimiento (Handshake de Tres Pasos)

1. **SYN:** El emisor envía su número de secuencia inicial (x).
2. **SYN + ACK:** El receptor confirma x+1 y envía su propio número inicial (y).
3. **ACK:** El emisor confirma y+1. Este proceso garantiza que ambos lados estén listos y acuerden los números de secuencia iniciales, protegiendo contra paquetes retrasados de conexiones antiguas.

Cierre y Terminación

- **Cierre Ordenado:** Utiliza un handshake modificado de tres pasos con bits **FIN**. Debido a que es Full Duplex, una dirección puede cerrarse mientras la otra continúa enviando datos.
- **Reinicio (Reset):** Mediante el bit **RST**, se provoca un aborto instantáneo de la conexión ante condiciones anormales, liberando recursos de inmediato.

Estado de Espera (Timed Wait)

TCP permanece en el estado **TIMED WAIT** durante el doble del tiempo de vida máximo del segmento (MSL). Esto asegura que los segmentos duplicados o retrasados de una conexión cerrada no interfieran con una nueva conexión.